



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 13 - Part 1

Protected Access & Friend Functions/Classes





THIS WEEK'S AGENDA

- 01. Recall – Access Specifiers
- 02. Protected Access
- 03. Friend Functions
- 04. Friend Classes
- 05. Summary





RECALL – ACCESS SPECIFIERS





RECALL – ACCESS SPECIFIERS

- Recall that in earlier classes we have studied **public** and **private** access specifiers.
- To recap, a **public** member of a class can be accessed and modified by any function within or outside the class.
- For safety, in general, we keep variables as **private** and provide access to them using accessor (getter) and mutator (setter) functions.
- Conversely, functions, in general are declared as **public** since these are mostly meant to be accessed by any function inside or outside the class.
- However, we've also seen that functions can also be declared as **private**.





PROTECTED ACCESS





PROTECTED ACCESS

- Another category of access level specifiers is the **protected** access, which we have seen in prior classes.
- Protected access is, in general, a feature of object oriented programming.
- Class members that are declared as **protected** can be accessed by any subclass (derived/child class) of that class as well.
- However, protected members cannot be accessed by classes outside of the inheritance hierarchy of the base class.





PROTECTED ACCESS – APPLICATION/USES

- The **protected** access specifier provides a middle ground between the private and public access specifiers.
- Recall that the private access specifier allowed access only within the class itself, and the public access specifier allowed access to everyone (including the main function).
- The **protected** access specifier is often used to allow subclasses to use or modify inherited members without exposing those members to other parts of the program.



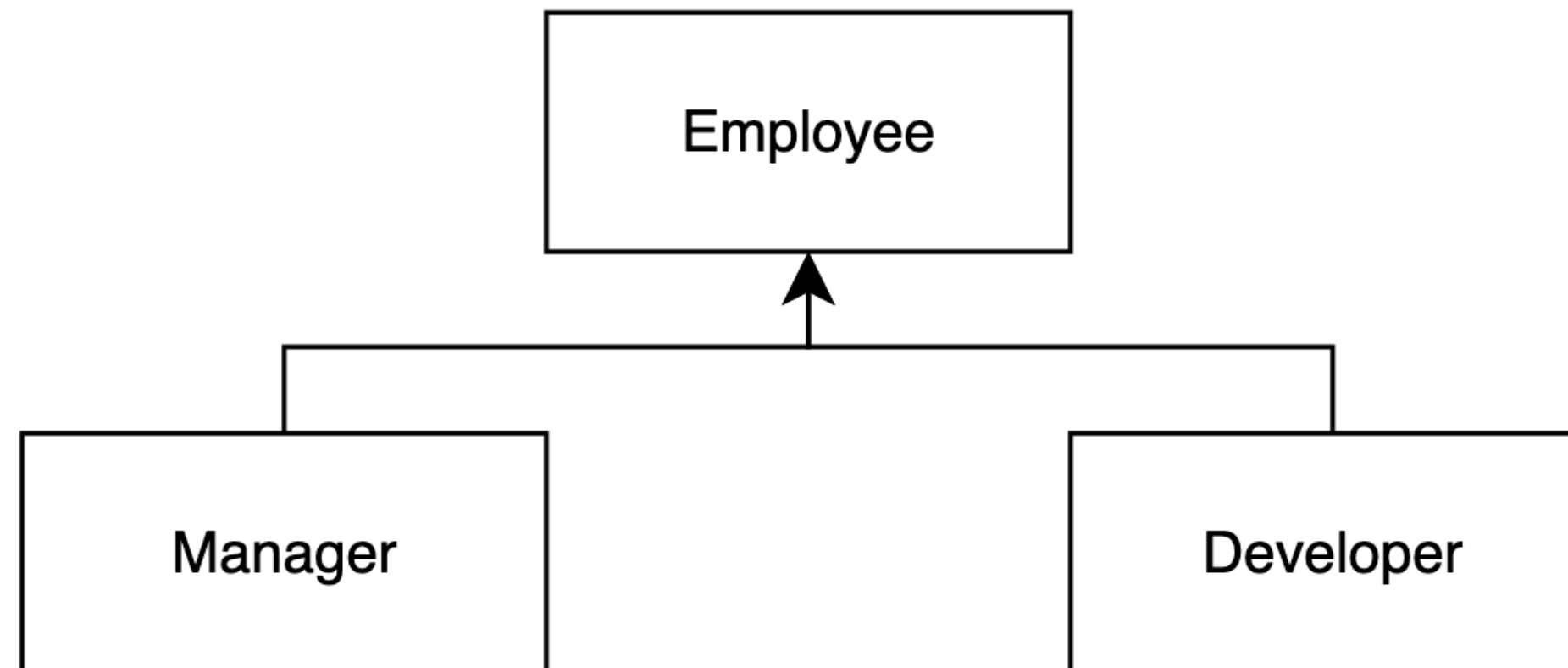


PROTECTED ACCESS – BENEFITS

- The two main benefits of using the **protected** access specifier are:
 - **Encapsulation with flexibility** – Members are hidden from the outside world (such as the main function) but are still available to all the sub-classes of the base class.
 - **Reusability** – Sub-classes can reuse and extend the functionality of the base class while maintaining the integrity of its data.



PROTECTED ACCESS – EXAMPLE





FRIEND FUNCTIONS





FRIEND FUNCTIONS

- **Friend functions** are functions that are not members of a class but are allowed access to the class's private and protected members.
- They provide us a way to extend the functionality of a class without making the function a member of the class itself.
- To declare a function as a **friend function**, the keyword *friend* is used inside the class definition.





FRIEND FUNCTIONS

- **Friend functions** are functions that are not members of a class but are allowed access to the class's private and protected members.
- They provide us a way to extend the functionality of a class without making the function a member of the class itself.
- To declare a function as a **friend function**, the keyword *friend* is used inside the class definition.





FRIEND FUNCTIONS

- The properties of data encapsulation and data hiding dictate that non-member functions (those that are not part of a class) should not be able to access the private and protected data members via an object of that class.
- However, there are situations where such rigid restrictions lead to considerable inconveniences.
- And in such situations a **friend function** can act as a bridge between two classes.
- For example:





FRIEND FUNCTIONS – EXAMPLE – SAME CLASS

```
#include <iostream>

class Cube {
    private:
        double length, width, height;
    public:
        Cube(double l, double w, double h) :
            length(l), width(w), height(h) {}

        friend double calculateVolume(Cube &c);
};
```

```
double calculateVolume(Cube &c) {
    return c.length * c.width * c.height;
}

int main() {
    Cube myCube(3.0, 4.0, 5.0);
    std::cout << "Volume of cube: " <<
        calculateVolume(myCube) << std::endl;
    return 0;
}
```



FRIEND FUNCTIONS – EXAMPLE – BRIDGE 2 CLASSES

```
#include <iostream>

class beta;

class alpha {
private:
    int data;
public:
    alpha() : data(5) {}
    friend int friendfunc(alpha &a, beta &b);
};
```

```
class beta {
private:
    int data;
public:
    beta() : data(2) {}
    friend int friendfunc(alpha &a, beta &b);
};

int friendfunc(alpha &a, beta &b) {
    return a.data + b.data;
}
```



SOME CONSIDERATIONS

- **Encapsulation** – While friend functions can access private members, we should use them sparingly so that we can maintain encapsulation principles.
- Overuse can lead to tightly coupled code.
- **Efficiency** – Friend functions can help to improve efficiency by eliminating the need for getters or setters for one-off operations.
- **Alternatives** – If a function is closely tied to a class, consider whether it should be a member instead of a friend.





FRIEND CLASSES





FRIEND CLASSES

- In C++, **friend classes** are classes that are permitted to access the private and protected members of another class.
- Declaring one class as a friend of another enables tight coupling between the two classes.
- This allows the **friend class** to access and modify the internals of the other class directly.
- However, the friendship relation is one-way which means that if **classA** declares **classB** as a friend, **classB** can access **classA's** private and protected members, but not vice versa unless explicitly declared.





FRIEND CLASSES – EXAMPLE

```
#include <iostream>

class CubeInspector;

class Cube {
private:
    double length, width, height;
public:
    Cube(double l, double w, double h) :
        length(l), width(w), height(h) {}

    friend class CubeInspector;
};
```

```
class CubeInspector {
public:
    double calculateVolume(const Cube& c) {
        return c.length * c.width * c.height;
    }
    void displayDimensions(const Cube& c) {
        std::cout << "Cube Dimensions: "
        << c.length << " x " << c.width << " x "
        << c.height << std::endl;
    }
};
```





FRIEND CLASSES – EXAMPLE

```
int main() {  
    Cube myCube(3.0, 4.0, 5.0);  
    CubeInspector cInspector;  
  
    cInspector.displayDimensions(myCube);  
    std::cout << cInspector.calculateVolume(myCube) << std::endl;  
    return 0;  
}
```





FRIEND CLASSES – BENEFITS

- Why should we use friend functions? One of the reasons is that it allows us to perform **encapsulation with collaboration**.
- As we saw in the earlier example, the ***CubeInspector*** class operates on ***Cube's*** private data without exposing it directly to the outside world.
- Another reason is that it provides **cleaner code** as it eliminates the need for excessive getters and setters, which could have cluttered the ***Cube*** class.
- In a real-world scenario, friend classes are useful in scenarios where one class is a natural manager of another class's data.
- Our example shows how friend class allow controlled interactions while keeping data encapsulation intact.





SUMMARY





SUMMARY

- Protected access allows access to a class's members to its sub-classes.
- Friend functions can act as a bridge between two classes and can access the private members of a class of which they are a friend.
- Friend classes allow for encapsulation of data with an aspect of collaboration between two classes.
- Friend classes are most useful when one class is being used to manage the other class's data.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 11





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

